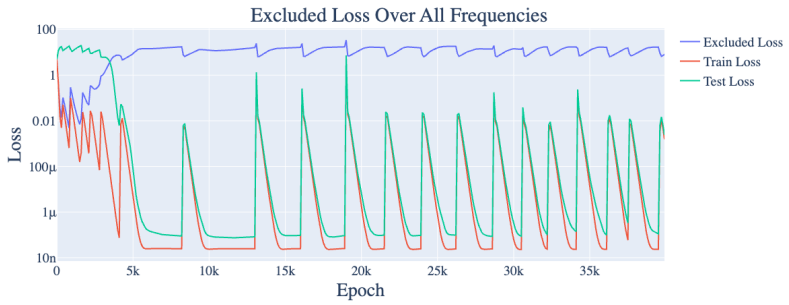

Figure 23: Training, test, and full excluded loss for a 2-layer version of the original architecture. One random seed chosen for readability.

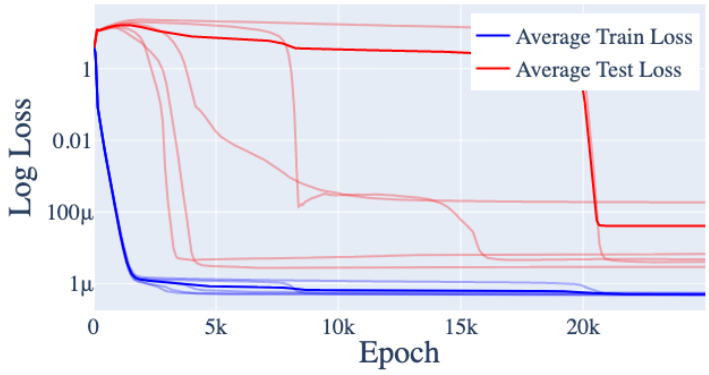

Figure 24: The training and test losses for $P = 53$ and all other hyperparameters except weight decay ($\gamma = 5$) the same as the main training run discussed in the paper. The averages are bold, and all contributing runs are partially transparent. Note that grokking occurs.

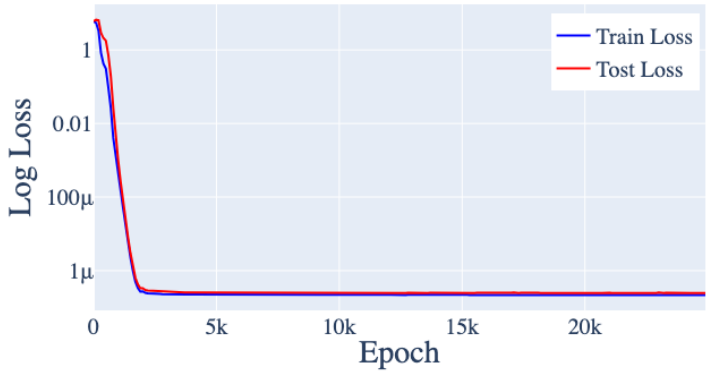

Figure 25: The training and test losses for $P = 401$ and all other hyperparameters the same as the main training run discussed in the paper. Grokking doesn't occur (the model generalizes immediately), even across a variety of weight decays.

this is because the memorization solution is significantly smaller (since there are only $53 \cdot 53$ total pairs), thereby requiring more aggressive weight decay for the generalizing solution to be favored.

For $P = 109$, we saw exactly the same behavior as with the mainline model.

For $P = 401$ (Figure 25), we could not get grokking, even by varying the weight decay parameter $\lambda \in \{0.3, 0.5, 1, 3, 5, 8\}$. Instead, the model immediately learns the generalizing solution. We believe this is because the amount of data seen by the model is greatly increased compared to the $P = 113$ case (from 30% of $113 \cdot 113$ pairs to 30% of $401 \cdot 401$ pairs), thereby favoring the generalizing solution from the start. We then trained 3 models each using 5%, 10%, 20% of the pairs of training data with $\lambda = 1$, and found that the models trained on 5% and 10% of the data immediately overfit and never generalized, while the models trained on 20% of the data also generalized immediately.

C.2.3 Generalizing models consistently use the Fourier Multiplication Algorithm

For each of the models in Appendix C.2.2 that achieve low test loss, we repeated the analysis performed in the mainline model, and summarize the results in Table 5. We list their key frequencies, Gini coefficients, and relevant FVEs. We find that every model trained with weight decay and that generalizes correctly implements some variation of the Fourier multiplication algorithm.

Interestingly, the embedding and unembedding matrices of the models trained with dropout are *not* sparse in the Fourier basis, and the logits for the $p = 0.2$ models are not as well explained by a sum of cosines as the other models (likely because the $p = 0.2$ models are simply worse at the task). We speculate that this is likely due to a combination of insufficient training epochs (as dropout models seem to take much longer to grok) and the inherent need for redundancy for networks trained via dropout.

As with the mainline model, we ignore the final skip connection (around the final MLP), as all of the generalizing models studied do not suffer significant performance penalties if the skip connection is zero or mean ablated (Table 6).

D Additional results on grokking

D.1 Both regularization and limited data are necessary for grokking

As discussed in Section 7 and Appendix C.2, the weight decay and the amount of data seem to have a strong effect on whether grokking occurs. To confirm this, we experiment with removing weight decay and varying the amount of data on 1-layer transformers. In Figure 26, we give the training, test, and full excluded loss for a typical training run with $\lambda = 0$ (no weight decay). As the figure shows, no grokking occurs, and excluded loss does not increase, suggesting that the model does not form the circuit for generalizing algorithm at all.

In Figure 20, we show the test loss curves for models trained with weight decay $\lambda = 1$ and on various fractions of the data. Though all the train losses are approximately the same—that is, they memorize at the same rate, models trained on smaller fractions of data take longer to grok.

In Figure 27, we display the test and train loss of models trained with $\lambda = 0.3$ and $\lambda = 3.0$. Smaller amounts of weight decay lead to slower grokking, while larger amounts of weight decay lead to faster grokking—on average, it takes around 3k epochs for models to grok with weight decay $\lambda = 0.3$, 5-10k epochs for the models to grok with weight decay $\lambda = 1.0$, and 20k epochs for the models to grok with weight decay $\lambda = 3.0$.

Finally, we test whether other forms of regularization can also induce grokking. We replaced weight decay with the following types of regularization while keeping all other hyperpameters the same:

1. **Dropout** We add dropout Srivastava et al. (2014) to the MLP neurons, with $p \in \{0.2, 0.5, 0.8\}$. That is, for each individual neuron, we set it to 0 with probability p during training, and also multiply the outputs of the other neurons by $\frac{1}{1-p}$.

Model	Test Loss	Gini(W_E)	Gini(W_L)	Key Frequencies	Logit FVE	MLP FVE
40% Training Data	$1.98 \cdot 10^{-7}$	0.76	0.79	[17, 43, 49, 55]	94.9%	83.3% [26.1%]
50% Training Data	$1.68 \cdot 10^{-7}$	0.75	0.77	[2, 17, 31, 41, 44]	91.2%	85.2% [28.2%]
60% Training Data	$1.23 \cdot 10^{-7}$	0.79	0.84	[2, 23, 34, 51]	96.4%	95.7% [1.4%]
70% Training Data	$9.85 \cdot 10^{-8}$	0.80	0.91	[14, 15, 26]	99.0%	98.9% [0.4%]
80% Training Data	$5.83 \cdot 10^{-7}$	0.62	0.80	[38, 41]	63.9%	94.1% [2.5%]
90% Training Data	$1.11 \cdot 10^{-7}$	0.79	0.88	[3, 26, 34, 43]	98.6%	98.7% [0.3%]
2 Layer Transformer	$9.54 \cdot 10^{-7}$	0.59	0.80	[14, 18, 29]	91.8%	95.2% [1.9%]
2 Layer Transformer	$4.41 \cdot 10^{-5}$	0.55	0.73	[7, 12, 35, 49]	86.1%	86.2% [6.4%]
2 Layer Transformer	$6.50 \cdot 10^{-2}$	0.66	0.80	[4, 9, 28]	88.5%	85.4% [5.9%]
2 Layer Transformer	$4.18 \cdot 10^{-2}$	0.56	0.76	[4, 5, 15, 54]	91.4%	81.2% [17.8%]
2 Layer Transformer	$1.75 \cdot 10^{-2}$	0.68	0.71	[3, 4, 13, 30, 38]	84.0%	71.9% [19.5%]
$P = 53$	$3.00 \cdot 10^{-4}$	0.61	0.68	[6, 9, 16, 21]	91.2%	90.2% [5.8%]
$P = 53$	$1.03 \cdot 10^{-4}$	0.56	0.72	[4, 13, 16]	94.8%	93.1% [6.4%]
$P = 53$	$1.21 \cdot 10^{-5}$	0.66	0.79	[13, 22, 23]	98.2%	97.6% [0.9%]
$P = 53$	$3.95 \cdot 10^{-6}$	0.66	0.74	[3, 14, 15]	88.5%	91.8% [4.6%]
$P = 53$	$5.56 \cdot 10^{-6}$	0.67	0.80	[10, 14, 22]	98.1%	98.3% [0.6%]
$P = 109$	$2.02 \cdot 10^{-7}$	0.76	0.83	[6, 7, 22, 25]	98.0%	97.3% [1.9%]
$P = 109$	$2.95 \cdot 10^{-7}$	0.69	0.82	[8, 14, 29, 32, 41]	95.2%	94.7% [2.3%]
$P = 109$	$1.66 \cdot 10^{-7}$	0.78	0.86	[13, 23, 39, 45]	98.5%	97.6% [0.9%]
$P = 109$	$2.50 \cdot 10^{-7}$	0.68	0.82	[8, 13, 32, 41]	96.8%	95.5% [2.3%]
$P = 109$	$2.77 \cdot 10^{-7}$	0.76	0.85	[29, 37, 38, 49]	97.9%	98.1% [0.8%]
Dropout $p = 0.2$	$2.65 \cdot 10^{-1}$	0.19	0.46	[1, 4, 7, 17, 22, 33, 40, 49, 55]	71.3%	65.0% [17.5%]
Dropout $p = 0.2$	$4.52 \cdot 10^{-1}$	0.19	0.46	[3, 8, 19, 28, 32, 34, 40, 44]	73.3%	71.4% [10.7%]
Dropout $p = 0.2$	$2.03 \cdot 10^{-1}$	0.20	0.45	[4, 5, 32, 38, 41, 44, 49, 50]	74.2%	71.1% [10.6%]
Dropout $p = 0.5$	$< 10^{-8}$	0.26	0.56	[1, 4, 26, 46, 47, 55]	89.4%	88.9% [3.5%]
Dropout $p = 0.5$	$2.01 \cdot 10^{-2}$	0.20	0.49	[16, 21, 35, 47, 53]	88.4%	88.4% [3.0%]
Dropout $p = 0.5$	$< 10^{-8}$	0.25	0.54	[1, 4, 7, 19, 29, 31, 42]	86.1%	85.6% [4.0%]

Table 5: For each of the models in Appendices C.2.3 and D.1 that generalizes to test data, we report the test loss, the Gini coefficients of the norms of the Fourier components of W_E and W_L (Section 5.1), the key frequencies of the network, and the fraction of variance in logits explained by a weighted sum of $\cos(w_k(a+b-c))$s over the key frequencies (Section 4.2).

In addition, we find the components u_k, v_k of W_L that correspond to cosines and sines of the key frequencies, and then report the average fraction of variance of $u_k^T \text{MLP}(a,b)$ and $v_k^T \text{MLP}(a,b)$ explained by a single term of form $\cos(w_k(a+b))$ or $\sin(w_k(a+b))$ respectively (Section 4.2). Numbers in square brackets represent the standard deviation. For 2 Layer models, we use the final layer MLP activations for $\text{MLP}(a,b)$.

We omit test accuracy because every model on this list except for the dropout $p = 0.2$ models achieves $> 99.95\%$ test accuracy, while the dropout $p = 0.2$ models achieve around 99.6% test accuracy.

Model Type	Loss	Accuracy	Ablated Loss	Ablated Acuracy
Varying Data Fraction	$1.83 \cdot 10^{-7}$ $(1.65 \cdot 10^{-7})$	100%	$7.74 \cdot 10^{-7}$ $(6.74 \cdot 10^{-7})$	100%
2 Layer Transformer	$1.97 \cdot 10^{-2}$ $(2.41 \cdot 10^{-2}$	99.6%	$4.63 \cdot 10^{-2}$ $(6.72 \cdot 10^{-2}$	98.7%
$P = 53$	$5.96 \cdot 10^{-5}$ $(8.91 \cdot 10^{-5})$	100%	$1.5 \cdot 10^{-4}$ $(2.70 \cdot 10^{-4})$	100%
$P = 109$	$1.94 \cdot 10^{-7}$ $(3.74 \cdot 10^{-8})$	100%	$6.53 \cdot 10^{-7}$ $(1.41 \cdot 10^{-7})$	100%
Dropout $p = 0.2$	0.215 (0.091)	99.7%	0.205 (0.075)	99.7%
Dropout $p = 0.5$	$4.68 \cdot 10^{-3}$ $(8.11 \cdot 10^{-3})$	100%	$3.6 \cdot 10^{-3}$ $(5.82 \cdot 10^{-3})$	100%

Table 6: We confirm that the skip connection around the final MLP layer is not important for performance by mean ablating the skip connection and computing loss and accuracy over the entire dataset for each problem, averaged over all runs. (We report the standard deviation of loss over the runs in parentheses.) While loss does increase a small amount, accuracy remains consistently high and the loss of the ablated model remains low. Results with zero ablations are also similar.